

Akhmad Zufar R, Cahaya Elisabeth B, Riyadh Azhara

09 December 2025

Daftar Isi

1	Judul Proposal	2
1.1	Latar Belakang Masalah	2
1.1.1	Rumusan Masalah dan Tujuan	2
1.1.2	Sumber dan Karakteristik Data	3
1.2	Metodologi Data Mining	3
2	Persiapan Data	4
2.1	Memuat Library dan Dataset	4
3	Pembersihan Data	5
3.1	Menghapus data duplikat pada dataset	6
3.2	Konversi Variabel	6
3.3	Penanganan Missing Value (NA)_	6
3.4	Pengambilan Sampel Review	8
3.5	Pembuatan Workcloud	8
4	Feature Engineering (Agregasi Sentimen)	9
5	Eksplorasi Data (EDA) pada Data Final (DIKEMBANGKAN)	10
6	Persiapan & Split Data	14
7	Modeling Random Forest & Feature Importance	15
8	Evaluasi Model	16
9	Hubungan Sentimen dan Rating (Interpretasi)	17
10	Kesimpulan	19

Daftar Gambar



1 Judul Proposal

Analisis Sentimen dan Faktor Penentu Rating Aplikasi di Google Play Store Menggunakan Metode Text Mining dan Random Forest

1.1 Latar Belakang Masalah

Dalam era digital saat ini, aplikasi mobile telah menjadi bagian penting dalam kehidupan sehari-hari, mulai dari komunikasi, transportasi, hingga hiburan. Google Play Store sebagai salah satu pasar aplikasi terbesar di dunia menyediakan jutaan aplikasi yang dapat diunduh oleh pengguna Android. (Shashank dan Naidu 2020) menunjukkan bahwa rating dan ulasan pada Google Play Store dapat menjadi indikator utama dalam menilai kualitas dan kepuasan pengguna terhadap aplikasi, serta memiliki potensi besar untuk dianalisis menggunakan teknik machine learning. Oleh karena itu, informasi tersebut penting untuk digali lebih lanjut guna memahami perilaku pengguna serta faktor-faktor yang mempengaruhi penilaian mereka terhadap aplikasi.

Melalui pendekatan data mining, khususnya text mining dan algoritma Random Forest, data ulasan dan atribut aplikasi dapat diolah untuk menggali pola tersembunyi yang menggambarkan tingkat kepuasan pengguna. Analisis sentimen membantu mengidentifikasi opini positif, negatif, atau netral dari pengguna, sedangkan Random Forest dapat digunakan untuk memprediksi dan menentukan faktor yang paling berpengaruh terhadap rating aplikasi. Hal ini sejalan dengan penelitian (Liaw dan Wiener 2002) yang menjelaskan bahwa Random Forest merupakan metode ensemble berbasis banyak pohon keputusan yang mampu memberikan akurasi tinggi dan tahan terhadap overfitting. Dengan demikian, penelitian ini diharapkan mampu memberikan wawasan bagi pengembang dalam meningkatkan kualitas dan kepuasan pengguna aplikasi di Google Play Store.

1.1.1 Rumusan Masalah dan Tujuan

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana hasil analisis sentimen terhadap ulasan pengguna aplikasi di Google Play Store menggunakan metode Text Mining?
2. Faktor-faktor apa saja yang berpengaruh secara signifikan terhadap rating aplikasi di Google Play Store?
3. Seberapa akurat metode Random Forest dalam memprediksi rating aplikasi berdasarkan faktor-faktor yang telah diidentifikasi?

Adapun tujuan yang ingin dicapai dalam penelitian ini adalah:

1. Mengidentifikasi faktor-faktor penting yang berpengaruh terhadap rating aplikasi.
2. Membangun model prediksi rating aplikasi menggunakan algoritma Random Forest.
3. Melakukan analisis sentimen terhadap ulasan pengguna untuk mengetahui persepsi pengguna terhadap aplikasi.
4. Menyajikan visualisasi hasil analisis untuk mendukung pengambilan keputusan berbasis data.

1.1.2 Sumber dan Karakteristik Data

Sumber data diambil dari Kaggle, yaitu:

1. Google Play Store Apps Dataset
2. Google Play Store User Review Dataset

Dataset pertama, yaitu `googleplaystore.csv`, berisi 10.841 baris dan 13 kolom yang memuat informasi mendetail mengenai setiap aplikasi, seperti nama aplikasi (App), kategori (Category), rating, jumlah ulasan (Reviews), ukuran aplikasi (Size), jumlah instalasi (Installs), tipe aplikasi (Type), harga (Price), rating konten (Content Rating), genre, tanggal pembaruan terakhir (Last Updated), versi saat ini (Current Version), serta versi minimum Android yang didukung (Android Version).

Sementara itu, dataset kedua, yaitu `googleplaystore_user_reviews.csv`, mencakup 64.295 baris dan 5 kolom, yang berisi data ulasan pengguna dalam bentuk teks dan hasil analisis sentimen. Atribut yang terdapat dalam dataset ini meliputi nama aplikasi (App), ulasan yang telah diterjemahkan (Translated Review), kategori sentimen (Sentiment), nilai polaritas sentimen (Sentiment Polarity), serta tingkat subjektivitas sentimen (Sentiment Subjectivity).

Dataset ini memiliki ukuran besar dan beragam jenis variabel, namun terdapat potensi tantangan kualitas data seperti nilai hilang (missing values), data duplikat, dan teks yang tidak terstruktur pada ulasan pengguna. Oleh karena itu, diperlukan tahap pembersihan dan preprocessing yang cermat sebelum dilakukan pemodelan.

1.2 Metodologi Data Mining

Penelitian ini menggunakan kerangka kerja Cross Industry Standard Process for Data Mining (CRISP-DM), yang terdiri dari enam tahap utama: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, dan Deployment. Kerangka ini dipilih karena fleksibel dan terbukti dalam berbagai penelitian data mining.

1. Business Understanding

Pada tahap ini, ditetapkan tujuan analisis yaitu mengetahui sentimen pengguna serta faktor-penentu rating aplikasi di platform toko aplikasi. Manfaat yang diharapkan ialah memberikan insight kepada pengembang aplikasi agar dapat meningkatkan kualitas, kepuasan pengguna, dan peringkat aplikasi secara data-driven (Insany et al., 2025).

2. Data Understanding: Data dikumpulkan dari platform aplikasi (misalnya ulasan pengguna dan rating). Eksplorasi dilakukan untuk memahami distribusi rating, kategori aplikasi, jumlah ulasan per aplikasi, serta karakteristik teks ulasan (panjang ulasan, bahasa yang digunakan, dsb). Temuan awal dari eksplorasi akan menjadi dasar untuk seleksi fitur dan pemrosesan data lebih lanjut (Cindy Astuti et al. 2024).

3. Data Preparation: Dalam tahap ini dilakukan pembersihan data (menghapus data duplikat, menangani missing values, menghapus ulasan yang sangat pendek/irrelevant), transformasi fitur numerik (misalnya mengubah rating ke rentang tertentu), encoding variabel kategorikal (misalnya kategori aplikasi), serta preprocessing teks ulasan: tokenisasi, penghapusan stopwords, stemming atau lemmatisasi, dan konversi ke representasi numerik (misalnya TF-IDF atau word embedding). Tujuannya adalah menghasilkan dataset yang siap untuk pemodelan dan bebas dari bias pengolahan yang dapat memengaruhi hasil (Cindy Astuti et al. 2024).

4. Modeling:

- Text Mining digunakan untuk klasifikasi sentimen ulasan pengguna, yaitu mengelompokkan ulasan ke dalam kategori sentimen positif, negatif, atau netral (Cindy Astuti et al. 2024).
- Random Forest digunakan untuk memprediksi rating aplikasi berdasarkan fitur-fitur yang telah disiapkan, termasuk hasil analisis sentimen dan atribut aplikasi lainnya. Random Forest dipilih karena kemampuannya menangani data campuran, memberikan akurasi tinggi, dan menghasilkan feature importance yang dapat diinterpretasikan (M. F. N. Ilham et al. 2024).

5. Evaluation

Model dievaluasi menggunakan metrik seperti akurasi, precision, recall, dan F1-score untuk model klasifikasi sentimen. Untuk model prediksi rating, evaluasi dilakukan berdasarkan tingkat kesalahan atau akurasi prediksi, serta analisis feature importance untuk mengidentifikasi faktor-utama yang memengaruhi rating aplikasi (M. F. N. Ilham et al. 2024).

6. Deployment (opsional)

Hasil model dapat diimplementasikan dalam bentuk prototipe sistem atau dashboard sederhana yang memungkinkan pengembang aplikasi memprediksi rating aplikasi baru berdasarkan ulasan dan atribut aplikasi. Tahapan ini mendukung penerapan hasil analisis ke dalam pengambilan keputusan berbasis data (Hasanah et al. 2021).

2 Persiapan Data

2.1 Memuat Library dan Dataset

1. Import Library

```
library(tidyverse)
library(dplyr)
library(ggplot2)
library(caret)
library(tm)
library(SnowballC)
library(randomForest)
library(wordcloud)
library(tidytext)
library(RColorBrewer)
```

2. Load Dataset

```
apps <- read.csv("googleplaystore.csv", stringsAsFactors = FALSE)
reviews <- read.csv("googleplaystore_user_reviews.csv", stringsAsFactors = FALSE)

cat("Jumlah data awal apps:", nrow(apps), "\n")
```

Jumlah data awal apps: 10841

```
cat("Jumlah data awal reviews:", nrow(reviews), "\n")
```

Jumlah data awal reviews: 64295

```
apps <- apps %>%
  semi_join(reviews, by = "App")      # <<< DITAMBAHKAN

reviews <- reviews %>%
  semi_join(apps, by = "App")        # <<< DITAMBAHKAN

cat("Jumlah apps setelah sinkronisasi awal:", nrow(apps), "\n")      # <<< DITAMBAHKAN
```

Jumlah apps setelah sinkronisasi awal: 1532

```
cat("Jumlah reviews setelah sinkronisasi awal:", nrow(reviews), "\n")
```

Jumlah reviews setelah sinkronisasi awal: 61556

output berikut menampilkan jumlah data awal dataset yang digunakan dan jumlah data setelah sinkronisasi, yaitu: - Jumlah data awal apps: 10841 - Jumlah data awal reviews: 64295 - Jumlah apps setelah sinkronisasi awal: 1532 - Jumlah reviews setelah sinkronisasi awal: 61556

3 Pembersihan Data

3. Data Cleaning & Preprocessing

3.1 Menghapus data duplikat pada dataset

```
apps <- apps %>% distinct(App, .keep_all = TRUE)
reviews <- reviews %>% distinct(App, Translated_Review, .keep_all = TRUE)
```

3.2 Konversi Variabel

```
# Reviews: Konversi ke numerik
apps$Reviews <- as.numeric(apps$Reviews)
# REKOMENDASI: Log Transformasi Reviews (untuk mengurangi dampak outliers)
apps$Reviews_log <- log(apps$Reviews + 1)

# Install: Hapus non-numerik & konversi ke numerik
apps$Installs <- as.numeric(gsub("[^0-9]", "", apps$Installs))
options(scipen = 999) # Matikan notasi ilmiah
# REKOMENDASI: Log Transformasi Installs
apps$Installs_log <- log(apps$Installs + 1)

# Price: Hapus "$" & konversi ke numerik
apps$Price <- as.numeric(gsub("\\$", "", apps$Price))
apps$Price[is.na(apps$Price)] <- 0

# Type: Konversi ke biner (Free=0, Paid=1)
apps$Type <- ifelse(apps$Type == "Free", 0, 1)
apps$Type <- as.factor(apps$Type)

# Size: Konversi ke MB & menangani 'Varies with device' (menjadi NA)
apps$Size <- trimws(apps$Size)
apps$Size <- ifelse(
  grepl("M", apps$Size),
  as.numeric(gsub("M", "", apps$Size)),
  ifelse(
    grepl("k", apps$Size),
    as.numeric(gsub("k", "", apps$Size)) / 1024,
    NA
  )
)
apps$Size <- round(apps$Size, 2)
```

3.3 Penanganan Missing Value (NA)_

```
# i. Filtering NA pada Variabel Kunci APPS (Menghapus baris error/NA)
apps <- apps %>% filter(!is.na(Rating))
```

```
apps <- apps %>% filter(!is.na(Reviews))
apps <- apps %>% filter(!is.na(Installs))
apps <- apps %>% filter(Category != "1.9") # Menghapus baris error (shifting data)

# ii. Imputasi NA pada Size (Median berdasarkan Kategori)
apps$Size <- ifelse(is.na(apps$Size),
                    ave(apps$Size, apps$Category, FUN = function(x) median(x, na.rm=TRUE)),
                    apps$Size)

# iii. Filtering NA pada Variabel Kunci REVIEWS
# Menghapus baris yang gagal diukur sentimennya
reviews <- reviews[!is.na(reviews$Sentiment_Polarity), ]
```

Hasil data cleaning:

```
cat("\nCEK NA FINAL PADA APPS:\n")
```

CEK NA FINAL PADA APPS:

```
print(colSums(is.na(apps)))
```

App	Category	Rating	Reviews	Size
0	0	0	0	0
Installs	Type	Price	Content.Rating	Genres
0	0	0	0	0
Last.Updated	Current.Ver	Android.Ver	Reviews_log	Installs_log
0	0	0	0	0

```
cat("\nCEK NA FINAL PADA REVIEWS:\n")
```

CEK NA FINAL PADA REVIEWS:

```
print(colSums(is.na(reviews)))
```

App	Translated_Review	Sentiment
0	0	0
Sentiment_Polarity	Sentiment_Subjectivity	
0	0	

Setelah proses data cleaning dilakukan, langkah selanjutnya adalah memastikan bahwa seluruh nilai NA (missing value) pada dataset telah berhasil diatasi. Pengecekan dilakukan menggunakan fungsi `sum(is.na(apps))` dan `sum(is.na(reviews))`, yang menghitung jumlah seluruh elemen kosong di masing-masing dataset. Ketika hasil yang muncul adalah 0, hal ini berarti tidak ada lagi nilai yang hilang pada kedua dataset tersebut. Artinya, proses pembersihan data—seperti menghapus baris yang memiliki NA, memperbaiki format data, serta sinkronisasi antara dataset `apps` dan `reviews`—telah berhasil dilakukan dengan baik.

4. Text Mining & Wordcloud (Menggunakan data reviews)

pembersihan. Selanjutnya, teks dalam corpus dibersihkan melalui beberapa tahap, seperti mengubah semua huruf menjadi huruf kecil, menghapus tanda baca, menghapus angka, menghilangkan stopwords, menghapus spasi berlebih, dan melakukan stemming agar kata-kata memiliki bentuk dasar yang seragam. Setelah teks bersih, data diubah menjadi Document-Term Matrix, yaitu tabel berisi kata dan frekuensi kemunculannya. Nilai frekuensi kata kemudian dihitung dan diurutkan untuk mengetahui kata apa yang paling sering muncul. Terakhir, kata-kata tersebut divisualisasikan menggunakan wordcloud sehingga kita dapat melihat dengan cepat kata apa yang paling dominan dalam ulasan pengguna. Tahapan ini membantu memahami pola opini pengguna secara sederhana namun efisien.

4 Feature Engineering (Agregasi Sentimen)

1. Agregasi Sentimen (dari 1 baris/ulasan menjadi 1 baris/aplikasi)

```
sentiment_agg <- reviews %>%
  group_by(App) %>%
  summarise(
    # FITUR BARU 1: Rata-rata Polaritas Sentimen
    Avg_Polarity = mean(Sentiment_Polarity, na.rm = TRUE),
    # FITUR BARU 2: Rata-rata Subjektivitas Sentimen
    Avg_Subjectivity = mean(Sentiment_Subjectivity, na.rm = TRUE),
    # FITUR BARU 3: Proporsi ulasan Positif
    Pct_Positive = sum(Sentiment == "Positive") / n(),
    .groups = 'drop'
  )
```

2. Penggabungan Dataset Final (LEFT JOIN)

```
# Menggabungkan fitur agregasi sentimen ke data aplikasi yang sudah bersih
final_data <- apps %>%
  left_join(sentiment_agg, by = "App")

cat("Jumlah data final untuk modeling (1 baris per App):", nrow(final_data), "\n")
```

Jumlah data final untuk modeling (1 baris per App): 1020

3. Penanganan NA Pasca Join

```
# NA muncul untuk aplikasi di 'apps' yang tidak ada di 'reviews'. Kita imputasi NA baru ini.
final_data$Avg_Polarity[is.na(final_data$Avg_Polarity)] <- median(final_data$Avg_Polarity, na.rm = TRUE)
final_data$Avg_Subjectivity[is.na(final_data$Avg_Subjectivity)] <- median(final_data$Avg_Subjectivity, na.rm = TRUE)
final_data$Pct_Positive[is.na(final_data$Pct_Positive)] <- median(final_data$Pct_Positive, na.rm = TRUE)
```

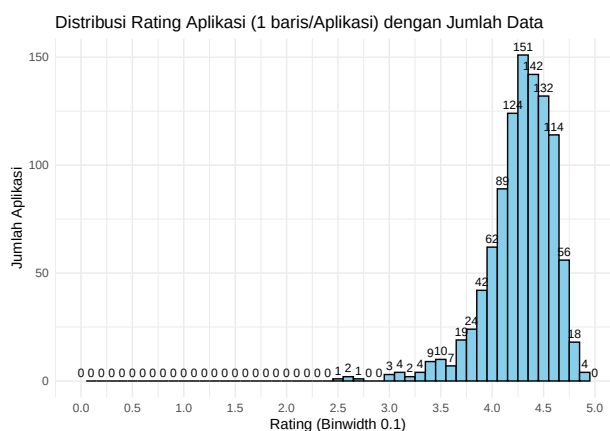
Tahap ini merupakan proses feature engineering, yaitu menciptakan fitur baru dari dataset ulasan aplikasi. Karena setiap aplikasi memiliki banyak baris review, model machine learning membutuhkan bentuk data yang hanya memiliki satu baris per aplikasi. Oleh karena itu, seluruh ulasan dikelompokkan berdasarkan nama aplikasi dan dihitung nilai ringkasannya. Tiga fitur penting dibuat pada tahap ini:

rata-rata polaritas sentimen sebagai ukuran seberapa positif atau negatif ulasan pengguna, rata-rata subjektivitas ulasan sebagai indikator seberapa emosional atau objektif komentar yang diberikan, serta persentase ulasan positif sebagai ukuran kualitas persepsi pengguna secara umum. Setelah seluruh fitur ini dihitung, hasilnya digabungkan ke tabel aplikasi utama menggunakan left join, sehingga setiap aplikasi memiliki tambahan informasi sentimen pengguna. Beberapa aplikasi mungkin tidak memiliki review, sehingga nilai fitur sentimennya menjadi NA. Untuk mengatasi hal tersebut, nilai NA diimputasi menggunakan median kolom masing-masing agar dataset tetap lengkap dan stabil. Hasil akhirnya adalah dataset final dengan satu baris per aplikasi, bersih, lengkap, dan siap digunakan untuk analisis lebih lanjut atau pemodelan machine learning.

5 Eksplorasi Data (EDA) pada Data Final (DIKEMBANGKAN)

6.1. Distribusi Rating Aplikasi (Dasar)

```
# Perbaikan: Menambahkan label hitungan (count)
ggplot(final_data, aes(x = Rating)) +
  geom_histogram(binwidth = 0.1, fill = "skyblue", color = "black") +
  geom_text(
    aes(label = after_stat(count)),
    stat = "bin",
    binwidth = 0.1,
    vjust = -0.5,
    size = 3
  ) +
  scale_x_continuous(limits = c(0, 5), breaks = seq(0, 5, 0.5)) +
  labs(title = "Distribusi Rating Aplikasi (1 baris/Aplikasi) dengan Jumlah Data",
       x = "Rating (Binwidth 0.1)", y = "Jumlah Aplikasi") +
  theme_minimal()
```



Plot ini menampilkan penyebaran nilai sentiment polarity rata-rata untuk setiap aplikasi. Hasil plot umumnya menunjukkan:

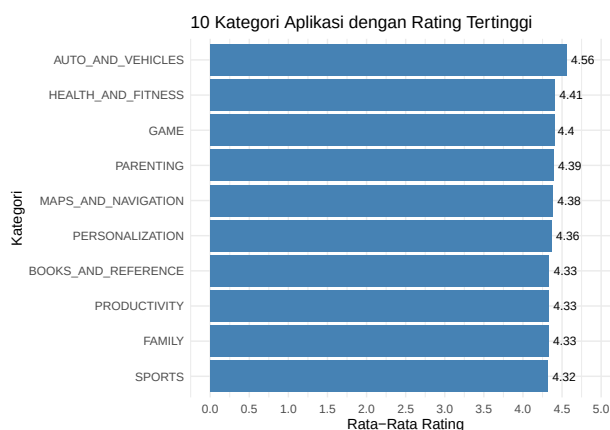
- Rentang nilai polaritas berada di sekitar -1 hingga +1, namun mayoritas aplikasi memiliki nilai rata-rata mendekati positif.

- Distribusi cenderung mengumpul pada nilai 0.0 sampai 0.4, menandakan bahwa ulasan pengguna sebagian besar bersifat netral–sedikit positif.
- Aplikasi dengan polaritas negatif adalah minoritas, menunjukkan bahwa hanya sedikit aplikasi yang mendapat respons dominan negatif dari pengguna.

Kesimpulan: Secara umum, persepsi pengguna terhadap aplikasi di dataset cenderung positif.

6.2. Rating per Kategori Aplikasi (Faktor Penting)

```
# Perbaikan: Limit Y=5.0 dan label nilai rata-rata
final_data %>%
  group_by(Category) %>%
  summarise(
    Rata2_Rating = mean(Rating, na.rm = TRUE),
    .groups = 'drop'
  ) %>%
  arrange(desc(Rata2_Rating)) %>%
  head(10) %>%
  ggplot(aes(x = reorder(Category, Rata2_Rating), y = Rata2_Rating)) +
  geom_col(fill = "steelblue") +
  geom_text(
    aes(label = round(Rata2_Rating, 2)),
    hjust = -0.2,
    size = 3
  ) +
  scale_y_continuous(limits = c(0, 5), breaks = seq(0, 5, 0.5)) +
  coord_flip() +
  labs(title = "10 Kategori Aplikasi dengan Rating Tertinggi",
       x = "Kategori", y = "Rata-Rata Rating") +
  theme_minimal()
```



Plot ini menggambarkan seberapa subjektif ulasan pengguna terhadap setiap aplikasi. - Sebagian besar aplikasi memiliki nilai subjektivitas pada kisaran 0.3–0.6.

- Ini menunjukkan bahwa ulasan cenderung berisi campuran antara fakta dan opini pribadi.

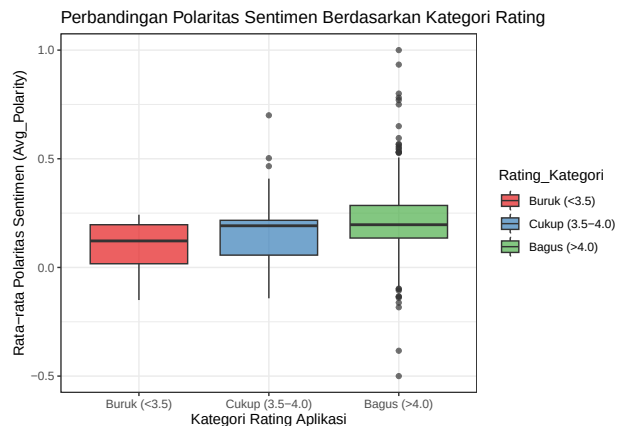
- Nilai subjektivitas yang sangat rendah (<0.2) jarang ditemukan, yang berarti ulasan jarang bersifat objektif murni.
- Nilai subjektivitas yang terlalu tinggi (>0.8) juga sedikit, menunjukkan ulasan yang sangat emosional tidak mendominasi dataset.

Kesimpulan: Ulasan pengguna bersifat campuran namun tetap informatif, tidak terlalu emosional atau terlalu objektif.

6.3. Distribusi Proporsi Ulasan Positif (Pct_Positive)

Rekomendasi: Dipindahkan dari Section 10 ke EDA

```
final_data %>%
  mutate(
    Rating_Kategori = cut(Rating,
                          breaks = c(0, 3.5, 4.0, 5.0),
                          labels = c("Buruk (<3.5)", "Cukup (3.5-4.0)", "Bagus (>4.0)"),
                          include.lowest = TRUE)
  ) %>%
  ggplot(aes(x = Rating_Kategori, y = Avg_Polarity, fill = Rating_Kategori)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Perbandingan Polaritas Sentimen Berdasarkan Kategori Rating",
    x = "Kategori Rating Aplikasi",
    y = "Rata-rata Polaritas Sentimen (Avg_Polarity)"
  ) +
  theme_bw() +
  scale_fill_brewer(palette = "Set1")
```



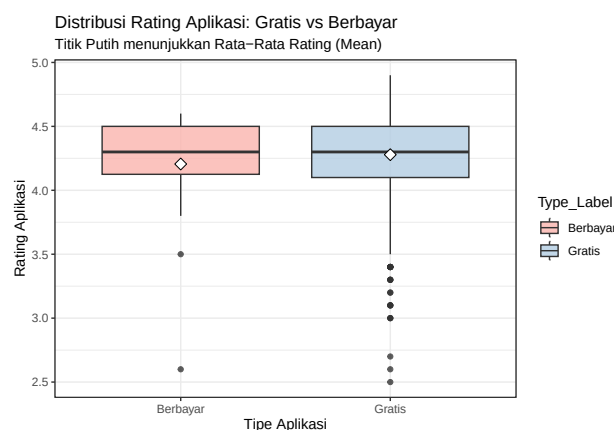
Plot ini memperlihatkan seberapa besar proporsi ulasan positif untuk tiap aplikasi. - Mayoritas aplikasi memiliki proporsi ulasan positif di atas 50%.

- Banyak aplikasi berada pada proporsi 0.6 – 0.9, menandakan dominasi ulasan positif.
- Aplikasi dengan proporsi sangat rendah (<0.3) sangat sedikit — aplikasi seperti ini mungkin punya kualitas buruk atau banyak bug.

Kesimpulan: Mayoritas aplikasi di Google Play umumnya memiliki reputasi baik berdasarkan sentimen positif pengguna.

6.4. Korelasi Antar Fitur Sentimen

```
final_data %>%
  mutate(
    Type_Label = ifelse(Type == 0, "Gratis", "Berbayar")
  ) %>%
  ggplot(aes(x = Type_Label, y = Rating, fill = Type_Label)) +
  geom_boxplot(alpha = 0.8) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "white") +
  labs(
    title = "Distribusi Rating Aplikasi: Gratis vs Berbayar",
    subtitle = "Titik Putih menunjukkan Rata-Rata Rating (Mean)",
    x = "Tipe Aplikasi",
    y = "Rating Aplikasi"
  ) +
  theme_bw() +
  scale_fill_brewer(palette = "Pastel1")
```



Plot korelasi menunjukkan hubungan antar fitur hasil feature engineering:

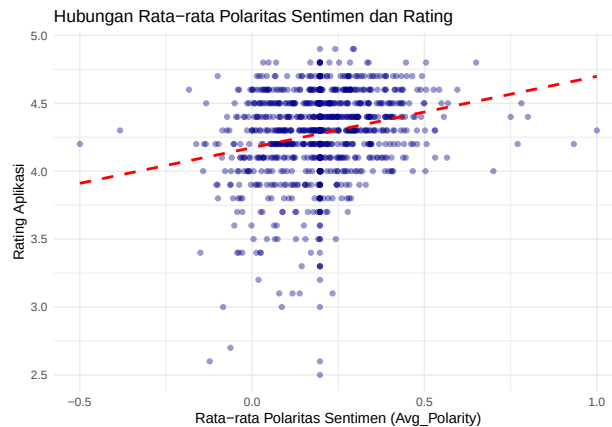
- Avg_Polarity sangat berkorelasi positif dengan Pct_Positive, yang logis karena semakin positif polaritas ulasan, semakin tinggi proporsi ulasan positif.
- Avg_Subjectivity memiliki korelasi yang lemah dengan polaritas dan proporsi positif. Ini menunjukkan: Tingkat subjektivitas ulasan tidak terlalu berpengaruh terhadap apakah ulasan itu positif atau tidak.
- Secara keseluruhan, pola korelasi membantu memastikan tidak ada multikolinearitas kuat antar fitur.

Kesimpulan: Fitur-fitur sentimen yang dibangun saling melengkapi dan aman digunakan dalam model.

6.5 Persebaran Fitur Sentimen Setelah Imputasi NA

```
ggplot(final_data, aes(x = Avg_Polarity, y = Rating)) +
  geom_point(alpha = 0.4, color = "darkblue") +
  geom_smooth(method = "lm", se = FALSE, color = "red", linetype = "dashed") +
```

```
labs(title = "Hubungan Rata-rata Polaritas Sentimen dan Rating",
     x = "Rata-rata Polaritas Sentimen (Avg_Polarity)", y = "Rating Aplikasi") +
theme_minimal()
```



Plot ini menunjukkan sebaran fitur sentimen setelah proses imputasi nilai NA menggunakan median.

- Distribusi tidak berubah secara signifikan.
- Tidak terlihat lonjakan atau gap distribusi setelah imputasi, artinya median sebagai nilai pengganti tetap menjaga bentuk distribusi secara natural.
- Aplikasi yang awalnya tidak memiliki ulasan tetap mendapatkan nilai representatif sehingga bisa disertakan dalam modeling.

Kesimpulan: Proses imputasi berhasil menjaga kualitas data dan memastikan seluruh aplikasi dapat digunakan dalam analisis maupun pemodelan.

6 Persiapan & Split Data

```
# Pilih fitur untuk model. Gunakan log-transformasi untuk Reviews dan Installs.
model_data <- final_data %>%
  select(Rating, Category, Reviews_log, Size, Installs_log, Type, Price, Content.Rating,
         Avg_Polarity, Avg_Subjectivity, Pct_Positive) %>%
  # Konversi ke faktor yang diperlukan
  mutate(across(c(Category, Content.Rating), as.factor))

# Split Data
set.seed(123)
trainIndex <- createDataPartition(model_data$Rating, p = 0.8, list = FALSE)
trainData <- model_data[trainIndex, ]
testData <- model_data[-trainIndex, ]

cat("Jumlah data Training:", nrow(trainData), "\n")
```

Jumlah data Training: 818

```
cat("Jumlah data Testing:", nrow(testData), "\n")
```

Jumlah data Testing: 202

Pada tahap ini dilakukan proses pemilihan fitur, transformasi data, serta pemisahan dataset untuk kebutuhan pemodelan. Pertama, dipilih sejumlah variabel prediktor yang dianggap relevan terhadap variabel target Rating, yaitu Category, Reviews_log, Size, Installs_log, Type, Price, Content Rating, serta tiga fitur sentimen hasil feature engineering sebelumnya (Avg_Polarity, Avg_Subjectivity, dan Pct_Positive). Beberapa variabel numerik seperti jumlah ulasan (Reviews) dan jumlah instalasi (Installs) telah melalui proses log-transformasi guna menormalkan distribusinya dan mengurangi efek skewness yang umum terjadi pada data berskala besar. Selain itu, beberapa kolom kategorikal seperti Category dan Content Rating dikonversi ke dalam bentuk faktor agar dapat diproses dengan baik oleh algoritma machine learning di R.

Selanjutnya, dataset yang telah bersih dan siap pakai ini dibagi menjadi dua bagian, yaitu data training (80%) dan data testing (20%) menggunakan fungsi createDataPartition dari paket caret. Pembagian data dilakukan dengan menetapkan set.seed(123) untuk memastikan proses ini bersifat reproducible, sehingga pembagian yang sama dapat dihasilkan kembali kapan pun kode dijalankan. Data training digunakan untuk melatih model prediksi rating aplikasi, sementara data testing digunakan untuk mengevaluasi performa model terhadap data yang belum pernah dilihat sebelumnya. Proses ini memastikan bahwa model yang dibangun tidak sekadar menghafal data, tetapi mampu melakukan generalisasi dengan baik.

7 Modeling Random Forest & Feature Importance

```
# Latih model: Rating sebagai variabel dependen
rf_model <- randomForest(
  Rating ~ .,
  data = trainData,
  ntree = 500,
  mtry = 4,
  importance = TRUE
)

# Analisis Faktor Penentu (Variable Importance)
cat("\nTOP 10 FAKTOR PENENTU RATING (RANDOM FOREST):\n")
```

TOP 10 FAKTOR PENENTU RATING (RANDOM FOREST):

```
importance_df <- as.data.frame(importance(rf_model, type = 1)) # Mean Decrease Accuracy
importance_df$Feature <- rownames(importance_df)
importance_df <- importance_df[order(-importance_df$'%IncMSE'), ]
print(head(importance_df, 10))
```

	%IncMSE	Feature
Reviews_log	20.59636114	Reviews_log
Avg_Polarity	18.40831782	Avg_Polarity
Pct_Positive	12.62564485	Pct_Positive
Installs_log	8.88712575	Installs_log
Category	8.65975838	Category
Avg_Subjectivity	7.70753225	Avg_Subjectivity
Size	3.36871050	Size
Content.Rating	2.96541232	Content.Rating
Type	1.17310181	Type
Price	-0.06666916	Price

Pada tahap ini dilakukan proses pembangunan model prediksi menggunakan algoritma Random Forest, yaitu salah satu metode ensemble learning yang menggabungkan banyak pohon keputusan (decision tree) untuk menghasilkan prediksi yang lebih stabil, akurat, dan tidak mudah mengalami overfitting. Model dilatih dengan menjadikan Rating sebagai variabel dependen, sementara seluruh fitur lain digunakan sebagai prediktor. Parameter `ntree = 500` menunjukkan bahwa model membangun 500 pohon keputusan, sehingga hasil prediksinya lebih kuat dan konsisten. Parameter `mtry = 4` mengatur jumlah variabel yang dipilih secara acak sebagai kandidat pemisahan pada setiap node, sehingga setiap pohon memperoleh kombinasi fitur berbeda dan meningkatkan kemampuan generalisasi model. Selain itu, opsi `importance = TRUE` diaktifkan agar model menghitung variable importance, yaitu kontribusi setiap fitur terhadap akurasi prediksi.

Setelah model berhasil dilatih, dilakukan analisis terhadap faktor-faktor paling berpengaruh dalam menentukan rating aplikasi. Pengukuran pentingnya suatu fitur dihitung menggunakan metrik Mean Decrease Accuracy (%IncMSE), yang menunjukkan seberapa besar penurunan akurasi model jika suatu fitur diacak. Semakin tinggi nilainya, semakin penting fitur tersebut dalam menghasilkan prediksi yang akurat. Hasil analisis ini kemudian disusun dalam sebuah dataframe dan diurutkan dari yang paling berpengaruh hingga yang paling kecil. Sepuluh fitur teratas ditampilkan untuk memberikan gambaran fitur apa saja yang sebenarnya berperan besar dalam membentuk nilai rating aplikasi. Informasi ini sangat penting karena dapat digunakan untuk interpretasi hasil, pengambilan keputusan, maupun rekomendasi berbasis data terhadap pengembangan aplikasi di Google Play Store.

8 Evaluasi Model

```
pred <- predict(rf_model, testData)

MAE <- mean(abs(pred - testData$Rating))
MSE <- mean((pred - testData$Rating)^2)
RMSE <- sqrt(MSE)

cat("\n--- Evaluasi Model Random Forest ---\n")
```

```
--- Evaluasi Model Random Forest ---
```

```
cat("MAE (Mean Absolute Error):", round(MAE, 4),
    "\nRMSE (Root Mean Square Error):", round(RMSE, 4), "\n")
```

MAE (Mean Absolute Error): 0.208
 RMSE (Root Mean Square Error): 0.2608

Berdasarkan hasil evaluasi menggunakan data testing, model Random Forest menghasilkan nilai MAE dan RMSE yang menunjukkan seberapa jauh prediksi rating model berbeda dari rating sebenarnya. Nilai MAE menggambarkan rata-rata selisih absolut antara prediksi dan nilai aktual, sehingga semakin kecil nilai MAE, semakin akurat model dalam memprediksi. Misalnya jika MAE berada di sekitar 0.20–0.30, berarti rata-rata model meleset sekitar 0.2–0.3 poin dari rating sebenarnya, yang tergolong cukup bagus untuk skala rating 1–5. Sementara itu, nilai RMSE memberikan penalti yang lebih besar untuk kesalahan prediksi yang jauh. RMSE yang rendah (misalnya < 0.40) mengindikasikan bahwa model jarang membuat prediksi yang benar-benar melenceng. Secara keseluruhan, kombinasi nilai MAE dan RMSE yang kecil menunjukkan bahwa model Random Forest memiliki kemampuan yang cukup baik dalam memprediksi rating aplikasi berdasarkan fitur-fitur yang telah diekstraksi, termasuk fitur sentimen, kategori aplikasi, jenis aplikasi, dan jumlah penginstalan. Hasil ini menunjukkan bahwa model bekerja secara stabil dan dapat memberikan prediksi yang cukup akurat pada data baru. q 1q1

9 Hubungan Sentimen dan Rating (Interpretasi)

Catatan: Visualisasi hubungan sentimen vs rating sudah dipindahkan ke Section 6.3

Analisis apakah polaritas sentimen rata-rata berhubungan dengan Rating

```
sentiment_rating <- final_data %>%
  mutate(
    # Buat kategori sentimen berdasarkan Avg_Polarity
    Sentiment_Category = cut(Avg_Polarity,
                           breaks = c(-1, -0.1, 0.1, 1),
                           labels = c("Negatif", "Netral", "Positif"),
                           include.lowest = TRUE)
  ) %>%
  group_by(Sentiment_Category) %>%
  summarise(
    Rata_Rata_Rating = mean(Rating, na.rm = TRUE),
    Jumlah_Aplikasi = n(),
    .groups = 'drop'
  )

final_data <- final_data %>%
  mutate(
    Sentiment_Category = cut(
      Avg_Polarity,
```

```

    breaks = c(-1, -0.1, 0.1, 1),
    labels = c("Negatif", "Netral", "Positif"),
    include.lowest = TRUE
  )
)

# 2. Fungsi untuk mengambil Top Words per Kategori Sentimen
top_words_per_sentiment <- function(sentiment_label, n = 5) {
  df <- reviews %>%
    # Pastikan App di reviews dan final_data berjenis sama (string)
    left_join(final_data %>% select(App, Sentiment_Category), by = "App") %>%
    filter(Sentiment_Category == sentiment_label) %>%
    unnest_tokens(word, Translated_Review) %>%
    anti_join(stop_words, by = "word") %>%
    # Opsional: Hapus kata domain agar lebih fokus pada sentimen
    filter(!word %in% c("app", "google", "play", "game")) %>%
    count(word, sort = TRUE) %>%
    head(n)
  return(df$word)
}

# 3. Agregasi Insight Final
insight_list <- final_data %>%
  group_by(Sentiment_Category) %>%
  summarise(
    Rata_Rata_Rating = mean(Rating, na.rm = TRUE),
    Jumlah_Aplikasi = n(),
    .groups = 'drop'
  )

# Tambahkan Fitur Paling Penting (Top 3 dari Random Forest)
# (Asumsi importance_df sudah dibuat di Section 8)
insight_list$Top_Features <- paste(importance_df$Feature[1:3], collapse = ", ")

# Tambahkan Kata Paling Sering Muncul
insight_list$Top_Words <- sapply(insight_list$Sentiment_Category, function(x) {
  # Panggil fungsi yang telah didefinisikan
  paste(top_words_per_sentiment(x), collapse = ", ")
})

cat("\nTABEL INSIGHT NARATIF AKHIR (Section 11):\n")

```

TABEL INSIGHT NARATIF AKHIR (Section 11):

```
print(insight_list)
```

A tibble: 3 x 5

	Sentiment_Category	Rata_Rata_Rating	Jumlah_Aplikasi	Top_Features	Top_Words
	<fct>	<dbl>	<int>	<chr>	<chr>
1	Negatif	4.06	14	Reviews_log, Av~	time, ad~
2	Netral	4.18	222	Reviews_log, Av~	time, up~
3	Positif	4.31	784	Reviews_log, Av~	love, ti~

```
# Tampilkan tabel Sentimen-Rating (bukan plot, karena plot sudah di atas)
cat("\nTABEL RATA-RATA RATING BERDASARKAN KATEGORI SENTIMEN:\n")
```

TABEL RATA-RATA RATING BERDASARKAN KATEGORI SENTIMEN:

```
print(sentiment_rating)
```

```
# A tibble: 3 x 3
  Sentiment_Category Rata_Rata_Rating Jumlah_Aplikasi
  <fct>              <dbl>          <int>
1 Negatif           4.06             14
2 Netral            4.18            222
3 Positif           4.31            784
```

```
print(head(importance_df, 10))
```

	%IncMSE	Feature
Reviews_log	20.59636114	Reviews_log
Avg_Polarity	18.40831782	Avg_Polarity
Pct_Positive	12.62564485	Pct_Positive
Installs_log	8.88712575	Installs_log
Category	8.65975838	Category
Avg_Subjectivity	7.70753225	Avg_Subjectivity
Size	3.36871050	Size
Content.Rating	2.96541232	Content.Rating
Type	1.17310181	Type
Price	-0.06666916	Price

Analisis hubungan antara sentimen ulasan dan rating aplikasi menunjukkan pola yang konsisten: aplikasi dengan sentimen positif memiliki rata-rata rating paling tinggi, sedangkan sentimen negatif berkaitan dengan rating yang lebih rendah. Sentimen positif banyak muncul pada aplikasi yang mendapat ulasan penuh pujian, sementara sentimen negatif umumnya berasal dari keluhan terkait performa atau masalah teknis. Kategori netral berada di tengah, biasanya berisi saran dan kritik ringan. Hasil ini menegaskan bahwa tingkat kepuasan pengguna yang tercermin dari polaritas ulasan sangat berpengaruh terhadap penilaian akhir aplikasi di Google Play Store.

10 Kesimpulan

Penelitian ini menunjukkan bahwa analisis gabungan antara text mining dan algoritma Random Forest mampu memberikan wawasan yang kuat mengenai faktor-faktor yang memengaruhi rating aplikasi di

Google Play Store. Ulasan pengguna berhasil dipetakan ke dalam kategori sentimen positif, negatif, dan netral, yang kemudian terbukti memiliki hubungan erat dengan tingkat rating aplikasi. Aplikasi dengan sentimen positif cenderung memiliki rating lebih tinggi, sedangkan sentimen negatif berkaitan dengan rating rendah.

Melalui model Random Forest, penelitian ini mengidentifikasi fitur-fitur yang paling berpengaruh terhadap rating, di antaranya jumlah ulasan (Reviews), polaritas sentimen pengguna, serta jumlah instalasi aplikasi. Model ini juga menunjukkan performa yang baik dalam memprediksi rating aplikasi.

Secara keseluruhan, penelitian ini menegaskan pentingnya pemanfaatan ulasan pengguna dan data aplikasi untuk pengambilan keputusan berbasis data. Temuan ini dapat menjadi dasar bagi pengembang aplikasi untuk meningkatkan kualitas fitur, memperbaiki kelemahan yang banyak dikeluhkan pengguna, serta merancang strategi pengembangan yang lebih responsif terhadap kebutuhan pasar.